

DAY 2

Customising Cursor

Rules · Skills · Hooks · Plugins · Subagents · MCP · Token Optimization

11:00 – 13:00 | Theory | 2 Hours

- › Module 3: Cursor Rules
- › Module 4: Skills
- › Module 5: Hooks
- › Module 6: Plugins, Subagents & MCP
- › Module 7: Token Optimization

Module 3

Cursor Rules

Persistence & consistency across sessions and team members

Module 3 — Cursor Rules

Persistence & consistency — the real value of rules

What Are Rules?

- Instructions Cursor automatically includes in prompts
- Written once in a file; Cursor loads them on the relevant condition
- Every session, every team member — without any action from the user

	Rules (.cursor/rules/*.mdc)
What they are	Text instructions injected into prompts
Location	.cursor/rules/
File format	Single .mdc file
Triggered by	alwaysApply, glob match, or agent decision
Can include scripts	No

Rules vs Stating Standards Once in Chat

Within a single chat thread, stating standards once IS equivalent — rules add no benefit there. Rules add genuine value only in situations chat context cannot handle.

Situation	Chat context	Rules
New chat / Ctrl+N (next day)	Context is gone — must re-establish manually	Loaded automatically every time
New team member opens project	They don't know what standards to set	Inherited from repo — zero onboarding needed
Long session (context eviction)	Standards stated early can silently disappear	Injected at system level — more reliably retained
File-type scoping	Not possible in a single opening statement	Auto Attached rules fire only for matching file types

Module 3 — Rule Types & File Format

Four types · .cursorrules (legacy) vs .cursor/rules/*.mdc (current)

Type	Trigger
Always	Loaded in every prompt
Apply to Specific Files	Loaded when glob-matching files are in context
Apply Intelligently	Agent decides based on rule description
Manual	Only when explicitly @-mentioned

alwaysApply	Description	Glob	Behaviour
True	-	-	Always included. Globs and description are ignored
False	-	Provided	Auto-attached when a matching file is in context.
False	Provided	Omitted	Agent reads the description and pulls the rule in when relevant
False	omitted	Omitted	Included only when you @-mention the rule in chat

Two Correct Forms — never mix them

Form 1 — Always rule (no file condition)

```
---
description: dbt and BigQuery SQL standards
alwaysApply: true  # loads every prompt
---
## SQL
- Dialect: BigQuery
- Use CTEs, never nested subqueries
```

Form 2 — Auto Attached (file-type scoped)

```
---
description: dbt standards for SQL/YAML files
globs:
  - "**/*.sql"
  - "**/*.yaml"
alwaysApply: false  # globs control trigger
---
```

Module 3 — Stream Examples & Scope

User Rules = personal (all projects) · Project Rules = team standard (commit to repo)

Stream A — dbt_standards.mdc (Always)

Cursor Settings → Rules → User Rules

Always rule: dbt_standards.mdc

SQL

- Dialect: BigQuery
- CTEs only, never nested subqueries
- Explicit column lists (no SELECT *)
- SAFE_DIVIDE() for division

dbt Naming

- Staging: stg_<source>_<entity>
- Intermediate: int_<description>
- Fact: fct_<fact> | Dim: dim_<entity>

Tests (every model)

- unique + not_null on PK
- accepted_values on status columns
- relationships test on FK columns

Stream B — python_standards.mdc (Always)

Cursor Settings → Rules → User Rules

Always rule: python_standards.mdc

Code Style

- Python 3.11+
- Type hints on all function signatures
- Max function length: 30 lines

Logging

- Use logging module, never print()
- Log row counts at each step

Error Handling

- Never bare except — catch specific types

File I/O

- if __name__ == "__main__": guard required
- pathlib.Path for all file paths
- index=False in DataFrame.to_csv()

Module 4

Skills

A portable package — the Agent opens templates, runs scripts, reads references

Module 4 — Skills

A portable package the Agent acts on — not just reads

From the Cursor documentation:

"A skill is a portable, version-controlled package that teaches agents how to perform domain-specific tasks. Skills can include scripts, templates, and references that agents may act on using their tools."

	Skills (.cursor/skills/<name>/)
What they are	Packaged task definitions with optional scripts/assets
Location	.cursor/skills/<skill-name>/ folder
File format	Folder with SKILL.md + optional subdirectories
Triggered by	Agent infers from context OR explicit /skill-name command
Can include scripts	Yes — scripts/, references/, assets/
Key distinction	Agent ACTS ON bundled resources — opens templates, runs scripts

Location	Scope
.agents/skills/	Project-level
.cursor/skills/	Project-level
~/.agents/skills/	User-level (global)
~/.cursor/skills/	User-level (global)

From Cursor Documentation:
For compatibility, Cursor also loads skills from Claude and Codex directories: .claude/skills/, .codex/skills/, ~/.claude/skills/, and ~/.codex/skills/.

Module 4 — Skill Folder Structure & SKILL.md

Folder name = skill name · SKILL.md directs Agent to use bundled resources

Skill Folder Structure

```
.cursor/skills/  
gen-staging-model/  
  SKILL.md          <- required: workflow + frontmatter  
  assets/  
    stg_template.sql <- Agent uses as starting template  
  references/  
    dbt_naming_conventions.md <- loaded on demand  
  scripts/  
    validate_output.py <- Agent runs after generating SQL
```

SKILL.md — What Makes It a Skill

```
---  
name: gen-staging-model  
description: Generate a dbt staging model.  
              Use when asked to create a staging  
              model or build a stg_layer.  
---  
  
## Workflow  
1. Use assets/stg_template.sql as starting structure  
2. Replace placeholders with actual table/columns  
3. Consult references/dbt_naming_conventions.md  
4. Run validator after generating:  
   python scripts/validate_output.py {filepath}  
5. Fix any warnings before finishing  
6. Generate schema.yml alongside the SQL
```

Field	Required	Purpose
name	Yes	Must match folder name exactly (lowercase, hyphens only)
description	Yes	Tells Agent when to apply — be specific, include trigger phrases
paths	No	Glob patterns to scope skill to matching file types only
disable-model-invocation	No	true = skill only runs via /skill-name command

Module 4 — Stream Examples

Agent auto-discovers skills from `.cursor/skills/` on startup · Also invoke with `/skill-name`

Stream A — gen-staging-model

Folder: `.cursor/skills/gen-staging-model/`

`SKILL.md`

`assets/stg_template.sql`

`references/dbt_naming_conventions.md`

`scripts/validate_output.py`

Invocation (automatic):

"Create a staging model for raw.orders"

→ Agent detects match, opens template,
runs `validate_output.py` after writing

Invocation (explicit):

`/gen-staging-model`

"Build staging model using `@source_schema.md`"

Stream B — gen-data-script

Folder: `.cursor/skills/gen-data-script/`

`SKILL.md`

`assets/script_template.py`

`scripts/format_check.py`

Invocation (automatic):

"Create a script that reads `sales_data.csv`",
→ Agent detects match, opens template,
runs `format_check.py` after writing

Invocation (explicit):

`/gen-data-script`

"Script to read CSV and save regional summary"

Module 5

Hooks

Spawned processes at agent loop events — JSON in, JSON out, can block actions

Module 5 — Hooks

12:10 – 12:40 | Spawned processes at agent loop events — not rules, real scripts

From the Cursor documentation:

"Hooks are spawned processes that communicate over stdio using JSON in both directions. They run before or after defined stages of the agent loop and can observe, block, or modify behavior."

	Hooks
What they are	Spawned scripts at agent loop events
Configured in	.cursor/hooks.json
Triggered by	Specific agent loop stages
Can block actions	Yes — exit code 2 blocks
Communication	Bidirectional — JSON in, JSON out
Runs	Outside the model — real process

hooks.json — Key Events for DE Work

Event	When it fires	Use for
afterFileEdit	After Agent writes/edits a file	Validate SQL or check Python style
preToolUse / postToolUse	Before/after any tool runs	Block or observe tool calls
beforeShellExecution	Before Agent runs a terminal command	Approve or deny shell commands
sessionStart	When new Agent session begins	Inject environment or context
stop	When Agent finishes	Trigger follow-up action

Module 5 — Hook Script Pattern & Exit Codes

Script reads JSON from stdin · writes JSON to stdout · exit code controls whether Cursor proceeds

How It Works

```
Agent writes stg_orders.sql
|
[afterFileEdit event]
|
Cursor spawns: python .cursor/hooks/validate_sql.py
|
stdin --> {"file_path": "models/stg_orders.sql", "edits": [...]}
|
script reads file_path, runs validation checks
|
stdout --> {"additional_context": "SELECT * found on line 4"}
|
appears in Agent chat --> Agent offers to fix
```

Hooks are not rules the Agent follows. Cursor spawns the script as a real process. The Agent has no role in triggering it — it always runs at the specified event.

Exit Codes

Exit code	Meaning
0	Success — Cursor uses the JSON output (additional_context shown in chat)
2	BLOCK the action — Cursor denies the file write entirely
Other	Hook failed — action proceeds silently (fail-open)

```
# Minimal hook pattern
import json, sys
from pathlib import Path

data = json.load(sys.stdin)
file_path = data.get("file_path", "")

if not file_path.endswith(".sql"):
    sys.exit(0)

issues = run_checks(Path(file_path))

print(json.dumps({
    "additional_context": "Issues: " + str(issues)
}))
sys.exit(0) # use sys.exit(2) to BLOCK
```

Module 5 — Stream Examples & Blocking

Stream A: SQL anti-pattern validation · Stream B: Python style check · Both use afterFileEdit event

Stream A — .cursor/hooks/validate_sql.py

.cursor/hooks.json:

```
{ "version": 1,
  "hooks": {
    "afterFileEdit": [
      { "command":
        "python .cursor/hooks/validate_sql.py" }
    ]
  }
}
```

validate_sql.py checks:

- SELECT * — use explicit column list
- CROSS JOIN — cartesian product risk
- Hardcoded date literals
- Missing {{ source() }} macro
- Unreplaced template placeholders

Stream B — .cursor/hooks/format_check_hook.py

.cursor/hooks.json:

```
{ "version": 1,
  "hooks": {
    "afterFileEdit": [
      { "command":
        "python .cursor/hooks/format_check_hook.py" }
    ]
  }
}
```

format_check_hook.py checks (stdlib only):

- print() calls — use logging.info()
- Bare except: — catch specific type
- Missing type hints on function args
- Missing if __name__ == "__main__" guard

DEMO

1. Show .cursor/hooks.json + .cursor/hooks/validate_sql.py (reads stdin JSON)
2. Ctrl+I (Agent mode) → "Generate staging model. Use SELECT * for columns."
3. After file written, Cursor spawns hook — watch additional_context appear in chat

Module 6

Plugins · Subagents · MCP

Bundle everything · Delegate tasks · Give Cursor new tools

Module 6 — Plugins: Distributable Bundles

12:40 – 13:00 | Rules + Skills + Hooks + MCP packaged for one-click install

Plugin Folder Structure

```
dbt-helper/
  .cursor-plugin/
    plugin.json      <- required manifest
  rules/
    dbt_standards.mdc  <- Always rule
  skills/
    gen-staging-model/
      SKILL.md        <- Skill
    assets/ references/ scripts/
  hooks.json          <- Hook config
  hooks/
    validate_sql.py    <- Hook script
  agents/
    dbt-reviewer.md    <- Subagent
  README.md
```

In a plugin, rules/ and skills/ sit at the PLUGIN ROOT — not inside .cursor/. The manifest tells Cursor where to find them.

plugin.json Manifest

```
{
  "name": "dbt-helper",
  "description": "dbt SQL tools – standards, staging skill, SQL validation hook",
  "version": "1.0.0",
  "author": { "name": "DE Team" }
}

# Only name is required.
# Components auto-discovered from
# rules/, skills/, hooks.json, agents/
```

Installing a Plugin

Method	Steps	Who gets it
Local testing/Local Repo	Copy to ~/.cursor/plugins/local/dbt-helper/, reload Cursor	You only
Cursor Marketplace	Publish at cursor.com/marketplace/publish	Anyone with Cursor
Team Marketplace (Teams/Enterprise Only)	Import GitHub repo via Customize → Plugins	Whole org, auto-updated on push

Module 6 — Subagents: Delegate with Context Isolation

Cursor calls these "Subagents" — not "Background Agents" · Defined in `.cursor/agents/` as markdown files

Built-in Subagents (No Config Needed)

Subagent	Purpose	Auto-used when
Explore	Codebase search and analysis	Agent needs to search before acting
Bash	Shell command sequences	Running terminal commands
Browser	Browser control via MCP	Web interaction needed

Custom Subagents — `.cursor/agents/*.md`

```
# .cursor/agents/dbt-reviewer.md
---
name: dbt-reviewer
description: Review a generated dbt model for
  correctness, naming, test coverage.
  Use proactively after generating any model.
model: inherit
readonly: true
---

Check: source()/ref() macros, no SELECT *,
surrogate key, schema.yml with tests...
```

Frontmatter Fields

Field	Default	Purpose
name	From filename	Identifier — lowercase, hyphens only
description	—	Tells parent agent when to delegate (be specific)
model	inherit	Parent's model, or a specific model ID
readonly	false	Prevents file edits — for analysis-only agents
is_background	false	Runs without blocking parent agent

Invocation Methods

- Automatic: Agent delegates based on task + description
 - "Generate a model then have it reviewed for correctness"
- Explicit: `/dbt-reviewer Review @models/stg_orders.sql`
- Parallel: "Generate models for all 3 sources in parallel"
 - → Agent launches three subagents simultaneously

Module 6 — MCP: Give Cursor New Tools

Model Context Protocol · stdio = Cursor auto-launches the server · HTTP/SSE = you run it, Cursor connects

Transport	Server management	Config format	Best for
stdio	Cursor auto-launches via command in mcp.json	"command" + "args"	Local Python/Node scripts — no manual start
SSE / HTTP	You run it independently; Cursor connects to URL	"url"	Remote or shared team servers

Configuring a Local Python MCP Server (stdio)

```
# .cursor/mcp.json
{
  "mcpServers": {
    "local-tools": {
      "command": "python",
      "args": [
        "labs/day2_lab/shared/simple_mcp_server.py"
      ],
      "env": {}
    }
  }
}

# Cursor reads this, spawns python automatically.
# No manual "python server.py" needed.
```

Lab MCP Server Tools

Tool	What it does
read_schema(table_name)	Reads {table_name}.md from project root — returns column info
list_project_files(ext)	Lists all files with given extension in working directory

Using in a prompt:

```
"Use read_schema to get the orders schema,
then generate a staging model for it."

-> Cursor calls read_schema("orders")
-> Tool reads orders.md from disk
-> Returns schema text to Cursor
-> Cursor generates staging model
    using real schema data — no pasting
```

DEMO

1. Show .cursor/mcp.json → Cursor Settings → MCP → local-tools connected

Keynote: "You did not run python server.py — Cursor launched it from mcp.json"

Module 6 — MCP: Give Cursor New Tools

Model Context Protocol · stdio = Cursor auto-launches the server · HTTP/SSE = you run it, Cursor connects

Configuring a Remote Python MCP Server (http) with OAuth

```
{
  "mcpServers": {
    "server-name": {
      "url": "http://localhost:3000/mcp",
      "headers": {
        "API_KEY": "value"
      }
    }
  },
  "oauth-server": {
    "url": "https://api.example.com/mcp",
    "auth": {
      "CLIENT_ID": "your-oauth-client-id",
      "CLIENT_SECRET": "your-client-secret",
      "scopes": ["read", "write"]
    }
  }
}
```

Module 7

Token Optimization

6 habits that take effect immediately · Apply in next 2h 40 min

Duration: 20 min — before the lab at 14:00

Module 7 — Token Optimization

14:00 – 14:20 | Before the lab · 6 habits that take effect immediately

What Is a Token?

- ~4 characters = 1 token
- Every prompt, @-reference, rule, and response consumes tokens
- Context window = hard cap — when full, earlier messages are pushed out

Note on Rules and Token Savings

Within a single conversation, stating standards once at the start is equally efficient — rules add no per-prompt token savings there. Rules save effort across sessions and team members: they survive Ctrl+N, are inherited by new colleagues, and are more reliably retained in long sessions.

6 Habits

- 1

New chat per task
Ctrl+N when switching tasks — never carry unrelated context forward
- 2

Scope file references
"In @file.py, the run() function on line 45..." not the whole file
- 3

Describe data, not rows
Column names + types beat 200 rows of CSV — zero extra reasoning value

Symptoms of Poor Token Management

Symptom	What's happening
AI "forgets" a rule from 10 messages ago	Earlier context pushed out of window
Response quality drops mid-session	Model working with truncated picture
Slow response on large codebase	Too many large files loaded at once
Cursor ignores rules mid-session	Rules crowded out by accumulated conversation

- 4

Rules = persistence
Value is survival across Ctrl+N and new teammates, not per-prompt savings
- 5

Scope @codebase
"Where is revenue aggregated?" beats "show me everything about orders"
- 6

Lightest mode for job
Ask < Plan < Multitask < Agent — match overhead to actual task

Module 7 — Day 2 Tools & Token Efficiency

All four tools built this morning are token-efficient by design

Day 2 Tool	Token benefit
.cursorrules / rules	Survive Ctrl+N and new colleagues — no re-establishing context each session
Skills (Agent Requested)	Loaded on demand only — never consumed when task does not match
Hooks (Auto Attached)	Scoped to file type — only loaded when relevant file is being edited
MCP read_schema tool	Fetches schema on demand — no need to paste DDL or schema markdown manually

Quick Reference Checklist

```
TOKEN EFFICIENCY — BEFORE EVERY PROMPT
New task?      -> Ctrl+N for fresh chat
Large file?    -> reference the section, not the whole file
Have data?     -> describe the schema, not the rows
Repeating context? -> rules persist across sessions; state once in chat is fine within a session
Searching codebase?-> ask a specific semantic question
Just need answer? -> Ask mode, not Agent
```